CS3320 Compilers 1

Assignment 2 – LLVM IR Exploration Report

Name: Krishnan R

Roll Number: CS24BTECH11036

Platform: WSL2 (Ubuntu)

Clang Version: Clang version 17.0.6

LLVM Version: 17.0.6

Introduction

This report delves into the LLVM compilation pipeline using Clang 17. It involves generating an IR (Intermediate Representation) for various constructs, analysing control flow and data structures, and studying optimizations (mainly -O0 and -O1).

Question 1

A) C Code in Q1a.c:

```
#include <stdio.h>
int a = 5; // (i) Global Integer
int main(){
        int b = 10; // (ii) A Local Integer
        int c = 3;  // (ii) A Local Integer
        int d = a + b * c; // (iii) A local Integer initialized to a + b * c
}
```

IR Code Snippets in Q1a.ll:

```
@a = dso_local global i32 5, align 4 ; (i) Global Integer a = 5
  %1 = alloca i32, align 4           ; (ii) allocate space for Local Integer b
  %2 = alloca i32, align 4           ; (ii) allocate space for Local Integer c
  %3 = alloca i32, align 4
  store i32 10, ptr %1, align 4      ; (ii) Initialize a Local Integer b = 10
  store i32 3, ptr %2, align 4       ; (ii) Initialize a Local Integer c = 3
  %4 = load i32, ptr @a, align 4     ; load a
  %5 = load i32, ptr %1, align 4     ; load b
  %6 = load i32, ptr %2, align 4     ; load c
  %7 = mul nsw i32 %5, %6            ; b * c
  %8 = add nsw i32 %4, %7            ; a + b * c
```

```
  store i32 %8, ptr %3, align 4      ; (iii) Store a + b * c in d
```

Explanation of why local variables appear as an alloca instruction rather than as named registers:

In LLVM IR, named registers follow the SSA (Static Single Assignment) form, meaning each register is assigned exactly once and does not have a memory address. On the other hand, C variables are real variables stored in memory (stack) which can be done using alloca (which allocates memory on stack to the variable), have an address, can be modified multiple times throughout the program. Therefore, Clang represents local variables using alloca instructions, which allocate space on the stack, and uses load and store instructions to read and modify their values. This allows LLVM IR to correctly model the mutable nature of variables in C.

B) C Code in Q1b.c:

```
#include <stdio.h>

int main(){
        int a = 2, b = 3;
        // (i) If-Else Condition
        if (a == 2){
                a = 2;
        }
        else{
                a = 2;
        }
        // (ii) Switch Case Condition
        switch(a){
                case 1: a = 2; break;
                case 2: a = 2; break;
                default: a = 2; break;
        }
        // (iii) For loop
        for (int i = a; i < 5; i++){
                a++;
        }
        // (iv) While loop
        while (a < 5){
                a++;
```

```
        }
        // (v) Do-While loop
        do {
                a++;
        } while ( a < 5);
        return 0;
}
```

IR Code Snippets in Q1b.ll:

Q1) Write C programs for each of the following constructs, generate the IR, and briefly describe how each is represented. For the loop variants, rewrite the same loop in all three forms and compare the IR.

`Preds = %x1 %x2…` means that the given block can be reach from blocks x1, x2,…

Explanations for each part:

i) if-else:

```
 %5 = load i32, ptr %2, align 4   ; load a
  %6 = icmp eq i32 %5, 2          ; check: a == 2
  br i1 %6, label %7, label %8    ; if (true) → go to block 7 else → go to block 8
7:                                ; preds = %0, Label 7 starts here
  store i32 2, ptr %2, align 4    ; do: a = 2 then
  br label %9                     ; go to next part (Block 9)
8:                                ; preds = %0, Label 8 starts here
  store i32 2, ptr %2, align 4    ; do: a = 2 then
  br label %9                     ; go to next part (Block 9)
```

ii) switch:

```
9:                                ; preds = %8, %7, Label 9 starts here
  %10 = load i32, ptr %2, align 4 ; load variable a into ptr 10
  switch i32 %10, label %13 [     ; switch(ptr 10 = a): else → go to 13
    i32 1, label %11              ; if a == 1 → go to 11
    i32 2, label %12              ; else if a == 2 → go to 12
  ]
11:                               ; preds = %9, Block 11 starts here
  store i32 2, ptr %2, align 4    ; Set a = 2
  br label %14                    ; Go to block 14
```

```
12:                               ; preds = %9, Label 12 starts here
  store i32 2, ptr %2, align 4    ; Set a = 2
  br label %14                    ; Go to block 14
13:                               ; preds = %9, Label 13 starts here
  store i32 2, ptr %2, align 4    ; Set a = 2
  br label %14                    ; Go to block 14
```

iii) for:

```
14:                                 ; preds = %13, %12, %11, Label 14 starts here
  %15 = load i32, ptr %2, align 4   ; read value from %2 (variable a), store it in
temporary %15
  store i32 %15, ptr %4, align 4    ; store that value into %4
  br label %16                      ; Jump to loop start
16:                                 ; preds = %22, %14, Label 16 starts here
  %17 = load i32, ptr %4, align 4   ; Load variable i into %17
  %18 = icmp slt i32 %17, 5         ; Check if i < 5
  br i1 %18, label %19, label %25 ; If i < 5, branch to loop body, else branch to
exit loop
19:                                 ; preds = %16, Label 19, loop body starts here
  %20 = load i32, ptr %2, align 4   ; Load variable a into %20
  %21 = add nsw i32 %20, 1          ; Store a++ into %21
  store i32 %21, ptr %2, align 4    ; Store the new value of a
  br label %22                      ; Branch to label 22
22:                                 ; preds = %19, Label 22 starts here
  %23 = load i32, ptr %4, align 4   ; Load variable i into %23
  %24 = add nsw i32 %23, 1          ; Store i++ into %24
  store i32 %24, ptr %4, align 4    ; Store the new value of i
  br label %16                      ; Branch to label 16
25:                                 ; preds = %16, Label 25 starts here
  br label %26                      ; Branch to label 26
```

iv) while:

```
26:                               ; preds = %29, %25, Label 26, while loop starts
here
  %27 = load i32, ptr %2, align 4 ; Load the value of variable a into %27
  %28 = icmp slt i32 %27, 5       ; Check if a < 5
```

```
  br i1 %28, label %29, label %32 ; Go to loop runner if a < 5, else go to exit
loop

29:                               ; preds = %26, Label 29, Loop runner starts here
  %30 = load i32, ptr %2, align 4 ; Load the value of variable a into %30
  %31 = add nsw i32 %30, 1        ; a++
  store i32 %31, ptr %2, align 4  ; Store the new value of a
  br label %26                    ; Branch to label 26 for the next iteration
32:                               ; preds = %26, Label 32, Loop exit starts here
  br label %33                    ; Branch to Label 33
```

v) do-while:

```
33:                         ; preds = %36, %32, Label 33, Do-While loop starts here
  %34 = load i32, ptr %2, align 4 ; Load the value of variable a into %34
  %35 = add nsw i32 %34, 1        ; a++
  store i32 %35, ptr %2, align 4  ; Store the new value of a
  br label %36                    ; Branch to label 36
36:                               ; preds = %33, Label 36 starts here
  %37 = load i32, ptr %2, align 4 ; Load the value of variable a into %37
  %38 = icmp slt i32 %37, 5       ; Check if a < 5
  br i1 %38, label %33, label %39 ; If a < 5, repeat the loop, else exit
39:                               ; preds = %36, Label 39, loop exit starts here
  ret i32 0                       ; int main() returns 0
```

Q2) i) Are for and while identical?

Explanation: Yes, the for and while loops are almost identical in their respective IR's. Both are implemented using a conditional (icmp) and branch (br) block, a loop body, branch instruction to either repeat the loop or exit the loop. The only difference is when we write the for loop in C, we explicitly define the loop iterator variable i, increment steps in the syntax while in the while loop, the compiler takes care of these details.

ii) Where does the condition check appear in the do-while IR compared to the other two?

Explanation: In both the for and while loops the condition check appears before entering the loop body while in the do-while loop they appear after the loop body finishes one iteration.

C) C Code in Q1c.c:

```
#include <stdio.h>

int square (int x){

        return x * x;
```

```
}
int main(){
        square(3);
        return 0;
}
```

IR Code Snippets in Q1c.ll:

Q) Locate and explain:

(i) the define and declare keywords

Explanation: The define keyword is present as define `dso_local i32 @square(i32 noundef %0) #0` (The definition of the int square(int) function) and as `define dso_local i32 @main() #0` (The definition of the main() function)

The declare keyword is present as `declare i32 @printf(ptr noundef, ...) #1` (Tells the compiler the function printf() definition is not present in the current file, since it is present in the external library stdio.h included as a header in the file)

The define keyword is used to define functions defined in the given file (like int square (int x) and int main()) itself while the declare keyword is used to declare functions whose definitions are not present in the current file but rather in external files (like printf() in the external library stdio.h declared at the start of the program) but called from the given file.

(ii) How arguments are passed by value and the return value is conveyed back

Explanation:

```
define dso_local i32 @square(i32 noundef %0) #0 {
  %2 = alloca i32, align 4          ; Allocate memory to store the argument in %0
  store i32 %0, ptr %2, align 4     ; Receives argument x and stores it in %2
  %3 = load i32, ptr %2, align 4
  %4 = load i32, ptr %2, align 4
  %5 = mul nsw i32 %3, %4
  ret i32 %5          ; Return the value after performing the required calculations
}
%2 = call i32 @square(i32 noundef 3) ; We pass the value 3 into the square()
function by value. The call instruction is used to receive the return value
conveyed back by the function.
```

When main() calls square(3), the value 3 is directed passed to the square() function and received as the parameter %0 inside the function. This value is then stored in %2 to which memory has been allocated and on which the required computation will be performed and the resultant value stored in %2 and returned back to the main function which called the

square function. The main function receives this value using the call instruction in register %2 which can then be used for further computation or passed to another function.

(iii) The role of the ret instruction.

Explanation:

```
ret i32 %5 ; return result of square
ret i32 0  ; return from main
```

The ret function is used to terminate a function, return the control to the caller while also returning value specified. In the square function ret i32 %5 returns the value x * x while in the main function ret i32 0 returns the value 0 to the OS.

D) C Code in Q1d.c:

```
#include <stdio.h>

int main(){

        // (i) Assigning Double to an Int (Narrowing)

        int a = 1;

        double b = 2.0;

        a = (int) b;

        // (ii) Assigning Int to a Long (Widening)

        int c = 3;

        long d = 4;

        d = (long) c;

        return 0;

}
```

IR Code Snippets in Q1d.ll:

Q) Identify the LLVM cast instructions used (fptosi, sext, zext, trunc) and explain when each variant is chosen.

(i) fptosi

Explanation:

```
%6 = load double, ptr %3, align 8

%7 = fptosi double %6 to i32
```

The instruction fptosi (floating point to signed integer) is used to convert a double to an integer (narrowing) where precision is lost in the process of converting a larger type (double) to a smaller type (int).

(ii) sext

Explanation:

```
%8 = load i32, ptr %4, align 4

%9 = sext i32 %8 to i64
```

The instruction sext (sign extension) is used to convert a 32 bit integer to a 64 bit integer (widening) where the sign is preserving while the value is extended.

(iii) zext

Explanation:

This one did not appear in my code. However, on further research I figured out that zext (zero extension) is used to extend unsigned integers by filling higher bits with 0's.

(iv) trunc

Explanation:

This one did not appear in my code. However, on further research I figured out that trunc (truncate) is used to convert a larger integer type to a smaller one by discarding higher bits.

Question 2

A) C Code in Q2a.c:

```
#include <stdio.h>

#include <stdlib.h>

int main(){

        int arr1[5];

        int *arr2 = (int*) malloc(5 * sizeof(int));

        for(int i = 0; i < 5; i++){

                arr1[i] = i;

        }

        int d = arr1[2];

        for(int i = 0; i < 5; i++){

                arr2[i] = i;

        }

        int e = arr2[2];

        return 0;

}
```

IR Code Snippets in Q2a.ll:

Q) In the generated IR, address:

(i) How does the compiler represent array access using getelementptr (GEP)? Show one GEP instruction in the report, explaining each operand.

Explanation:

The getelementptr (GEP) instruction is used to compute the address of an element in an array. GEP does not perform memory access, it only computes addresses.

Ex: `%16 = getelementptr inbounds [5 x i32], ptr %2, i64 0, i64 %15`

`[5 x i32]` → Array type (5 integers)

`ptr %2` → Base address of array

`i64 0` → First index (array itself)

`i64 %15` → Element index (i)

`arr1[i]` → address = base + i * sizeof(int) → GEP Computes this address

(ii) How is the array itself allocated in IR? What does the alloca instruction look like for an array type, and how does it differ from a scalar alloca?

Explanation:

The alloca instruction looks like `%2 = alloca [5 x i32], align 16` for an array type.

The alloca instruction allocates memory for the array on the stack. Here, the `[5 x i32]` indicates that the memory allocated is equal to the memory size of 5 integers.

On the other hand, the scalar alloca looks like `%1 = alloca i32, align 4` where the memory allocated is equal to the memory size of 1 integer.

(iii) Allocate one array statically and another one dynamically (using malloc), and identify the differences in the generated IR.

Explanation:

Static Allocation: `%2 = alloca [5 x i32], align 16`

Memory equal to the memory size of 5 integers is statically allocated on the stack for the array arr1.

Dynamic Allocation:
```
declare noalias ptr @malloc(i64 noundef) #1
%8 = call noalias ptr @malloc(i64 noundef 20)
store ptr %8, ptr %3
```

Since malloc() is an external function, a declare instruction is used.

Memory equal to the memory size of 5 integers (5 integers * 4 bytes = 20 bytes) is dynamically allocated on the heap for the array arr2 using malloc which returns a pointer %8 whose value is then stored in ptr %3.

LLVM IR represents array access using the getelementptr instruction, which computes addresses based on indices. Static arrays are allocated on the stack using alloca, while dynamic arrays are allocated on the heap using malloc. This difference is reflected in the IR through distinct allocation mechanisms and pointer types used in element access.

B) i) C Code in Q2bi.c:

```
#include <stdio.h>
struct Point{
        int x;
        int y;
};
int main(){
        struct Point p;
        p.x = 1;
        p.y = 2;
        return 0;
}
```

ii) C Code in Q2bii.cpp:

```
class Point{
public:
        int x;
        int y;
        // Constructor
        Point(){
                x = 0;
                y = 0;
        }
};
int main(){
        Point p;
        return 0;
}
```

IR Code Snippets in Q2bi.ll and Q2bii.ll:

Q) In the IR for both:

(i) Locate the type definition and state how member alignment is encoded.

Explanation:

Type definition of struct Point{ int x; int y; }: `%struct.Point = type { i32, i32 }`

Type definition of class Point{ int x; int y; }: `%class.Point = type { i32, i32 }`

The type definition shows that the struct/class Point consists of two i32 members (x and y) stored sequentially in memory.

```
%2 = alloca %struct.Point, align 4
store i32 1, ptr %3, align 4
```

Member alignment is not specified in the type definition but is specified during memory allocation (alloca), and memory access (store, load) using (align) keyword. Each i32 is aligned to 4 bytes.

(ii) Are there structural differences in the type layout between struct and class? How does the constructor appear in the class IR?

Explanation:

```
%struct.Point = type { i32, i32 }
```

```
%class.Point = type { i32, i32 }
```

There are no structural differences in the type layout between struct and class.

```
call void @_ZN5PointC2Ev(ptr %2)
```

```
define void @_ZN5PointC2Ev(ptr %0)
```

Constructors appear as normal functions in LLVM IR. The object pointer (this: %0) is passed as an argument, and the constructor initializes the fields using getelementptr and store instructions. Initializing happens with a constructor call with address of p (Here %2 in the call function).

(iii) How is accessing a member of a class or a struct different from accessing an element from an array?

Explanation:

Struct/Class access: `%3 = getelementptr inbounds %struct.Point, ptr %2, i32 0, i32 0`
`%4 = getelementptr inbounds %struct.Point, ptr %2, i32 0, i32 1`

Array access: `%15 = getelementptr inbounds [5 x i32], ptr %2, i64 0, i64 %index`

In Struct/Class access, we can access only fixed field indices (compile time constants). The GEP type is `%struct.Point` while in Array access we can access variable field indices (i, runtime values). The GEP type is `[5 x i32]`.

Question 3

C Code in input.c:

```
int ternary_example (int a, int b) {
        return (a > b) ? (a * 2) : (b + 1);
}
int main(){
        return 0;
```

```
}
```

A) i) Output:

```
int 'int'        [StartOfLine]  Loc=<input.c:1:1>
identifier 'ternary_example'     [LeadingSpace] Loc=<input.c:1:5>
l_paren '('      [LeadingSpace] Loc=<input.c:1:21>
int 'int'               Loc=<input.c:1:22>
identifier 'a'   [LeadingSpace] Loc=<input.c:1:26>
comma ','               Loc=<input.c:1:27>
int 'int'        [LeadingSpace] Loc=<input.c:1:29>
identifier 'b'   [LeadingSpace] Loc=<input.c:1:33>
r_paren ')'             Loc=<input.c:1:34>
l_brace '{'      [LeadingSpace] Loc=<input.c:1:36>
return 'return'  [StartOfLine] [LeadingSpace]   Loc=<input.c:2:2>
l_paren '('      [LeadingSpace] Loc=<input.c:2:9>
identifier 'a'          Loc=<input.c:2:10>
greater '>'      [LeadingSpace] Loc=<input.c:2:12>
identifier 'b'   [LeadingSpace] Loc=<input.c:2:14>
r_paren ')'             Loc=<input.c:2:15>
question '?'     [LeadingSpace] Loc=<input.c:2:17>
l_paren '('      [LeadingSpace] Loc=<input.c:2:19>
identifier 'a'          Loc=<input.c:2:20>
star '*'         [LeadingSpace] Loc=<input.c:2:22>
numeric_constant '2'     [LeadingSpace] Loc=<input.c:2:24>
r_paren ')'             Loc=<input.c:2:25>
colon ':'        [LeadingSpace] Loc=<input.c:2:27>
l_paren '('      [LeadingSpace] Loc=<input.c:2:29>
identifier 'b'          Loc=<input.c:2:30>
plus '+'         [LeadingSpace] Loc=<input.c:2:32>
numeric_constant '1'     [LeadingSpace] Loc=<input.c:2:34>
r_paren ')'             Loc=<input.c:2:35>
semi ';'                Loc=<input.c:2:36>
r_brace '}'      [StartOfLine]  Loc=<input.c:3:1>
int 'int'        [StartOfLine]  Loc=<input.c:5:1>
identifier 'main'        [LeadingSpace] Loc=<input.c:5:5>
l_paren '('             Loc=<input.c:5:9>
```

```
r_paren ')'             Loc=<input.c:5:10>

l_brace '{'             Loc=<input.c:5:11>

return 'return'  [StartOfLine] [LeadingSpace]   Loc=<input.c:6:2>

numeric_constant '0'     [LeadingSpace] Loc=<input.c:6:9>

semi ';'                Loc=<input.c:6:10>

r_brace '}'      [StartOfLine]  Loc=<input.c:7:1>

eof ''          Loc=<input.c:7:2>
```

Types of tokens

1) Keywords:

```
int ‘int’. Extra info: int ‘int’ [StartOfLine]  Loc=<input.c:5:1> implies line 5,
column 1 at the start of a new line.
return ‘return’. Extra info: return 'return'  [StartOfLine] [LeadingSpace]
Loc=<input.c:6:2> implies line 6, column 2 at the start of a new line along with a
Leading Whitespace.
```

2) Identifiers

```
identifier 'ternary_example'. Extra info: identifier 'ternary_example'
[LeadingSpace] Loc=<input.c:1:5> implies line 1, column 5 with a Leading
Whitespace.
identifier 'a'. Extra info: identifier 'a'          Loc=<input.c:2:20> implies
line 2, column 20.
identifier 'b'. Extra info: identifier 'b'          Loc=<input.c:2:30> implies
line 2, column 30.
identifier 'main’. Extra info: identifier 'main'        [LeadingSpace]
Loc=<input.c:5:5> implies line 5, column 5 with a Leading Whitespace.
```

3) Operators

```
greater '>'. Extra info: greater '>'      [LeadingSpace] Loc=<input.c:2:12>
implies line 2, column 12 with a Leading Whitespace.
question '?'. Extra info: question '?'     [LeadingSpace] Loc=<input.c:2:17>
implies line 2, column 17 with a Leading Whitespace.
colon ':'. Extra info: colon ':'        [LeadingSpace] Loc=<input.c:2:27> implies
line 2, column 27 with a Leading Whitespace.
plus '+'. Extra info: plus '+'         [LeadingSpace] Loc=<input.c:2:32> implies
line 2, column 32 with a Leading Whitespace.
star '*'. Extra info: star '*'         [LeadingSpace] Loc=<input.c:2:22> > implies
line 2, column 22 with a Leading Whitespace.
```

4) Constants

```
numeric_constant '2'. Extra info: numeric_constant '2'     [LeadingSpace]
Loc=<input.c:2:24> implies line 2, column 24 with a Leading Whitespace.
numeric_constant '1'. Extra info: numeric_constant '1'     [LeadingSpace]
Loc=<input.c:2:34> implies line 2, column 34 with a Leading Whitespace.
numeric_constant '0'. Extra info: numeric_constant '0'     [LeadingSpace]
Loc=<input.c:6:9> implies line 6, column 9 with a Leading Whitespace.
```

5) Punctuation / Symbols

```
l_paren '('. Extra info: l_paren '('      [LeadingSpace] Loc=<input.c:1:21>
implies line 1, column 21 with a Leading Whitespace.
r_paren ')'. Extra info: r_paren ')'             Loc=<input.c:1:34> implies line
1, column 34.
l_brace '{'. Extra info: l_brace '{'      [LeadingSpace] Loc=<input.c:1:36>
implies line 1, column 36 with a Leading Whitespace.
r_brace '}'. Extra info: r_brace '}'      [StartOfLine]  Loc=<input.c:3:1> implies
line 1, column 36 at the start of a new line.
comma ','. Extra info: comma ','               Loc=<input.c:1:27> implies line 1,
column 27.       semi ';'. Extra info: semi ';'               Loc=<input.c:2:36>
implies line 2, column 36.
```

ii) Output: `TranslationUnitDecl 0x57cc6aa485d8 <<invalid sloc>> <invalid sloc>`

```
|-TypedefDecl 0x57cc6aa48e08 <<invalid sloc>> <invalid sloc> implicit __int128_t
'__int128'

| `-BuiltinType 0x57cc6aa48ba0 '__int128'

|-TypedefDecl 0x57cc6aa48e78 <<invalid sloc>> <invalid sloc> implicit __uint128_t
'unsigned __int128'

| `-BuiltinType 0x57cc6aa48bc0 'unsigned __int128'

|-TypedefDecl 0x57cc6aa49180 <<invalid sloc>> <invalid sloc> implicit
__NSConstantString 'struct __NSConstantString_tag'

| `-RecordType 0x57cc6aa48f50 'struct __NSConstantString_tag'

|   `-Record 0x57cc6aa48ed0 '__NSConstantString_tag'

|-TypedefDecl 0x57cc6aa49228 <<invalid sloc>> <invalid sloc> implicit
__builtin_ms_va_list 'char *'

| `-PointerType 0x57cc6aa491e0 'char *'

|   `-BuiltinType 0x57cc6aa48680 'char'

|-TypedefDecl 0x57cc6aa49520 <<invalid sloc>> <invalid sloc> implicit
__builtin_va_list 'struct __va_list_tag[1]'

| `-ConstantArrayType 0x57cc6aa494c0 'struct __va_list_tag[1]' 1

|   `-RecordType 0x57cc6aa49300 'struct __va_list_tag'

|     `-Record 0x57cc6aa49280 '__va_list_tag'

|-FunctionDecl 0x57cc6aaa8868 <input.c:1:1, line:3:1> line:1:5 ternary_example
'int (int, int)'

| |-ParmVarDecl 0x57cc6aaa8700 <col:22, col:26> col:26 used a 'int'

| |-ParmVarDecl 0x57cc6aaa8780 <col:29, col:33> col:33 used b 'int'

| `-CompoundStmt 0x57cc6aaa8b88 <col:36, line:3:1>

|   `-ReturnStmt 0x57cc6aaa8b78 <line:2:2, col:35>

|     `-ConditionalOperator 0x57cc6aaa8b48 <col:9, col:35> 'int'

|       |-ParenExpr 0x57cc6aaa89f8 <col:9, col:15> 'int'

|       | `-BinaryOperator 0x57cc6aaa89d8 <col:10, col:14> 'int' '>'
```

```
|       |   |-ImplicitCastExpr 0x57cc6aaa89a8 <col:10> 'int' <LValueToRValue>

|       |   | `-DeclRefExpr 0x57cc6aaa8968 <col:10> 'int' lvalue ParmVar
0x57cc6aaa8700 'a' 'int'

|       |   `-ImplicitCastExpr 0x57cc6aaa89c0 <col:14> 'int' <LValueToRValue>

|       |     `-DeclRefExpr 0x57cc6aaa8988 <col:14> 'int' lvalue ParmVar
0x57cc6aaa8780 'b' 'int'

|       |-ParenExpr 0x57cc6aaa8a90 <col:19, col:25> 'int'

|       | `-BinaryOperator 0x57cc6aaa8a70 <col:20, col:24> 'int' '*'

|       |   |-ImplicitCastExpr 0x57cc6aaa8a58 <col:20> 'int' <LValueToRValue>

|       |   | `-DeclRefExpr 0x57cc6aaa8a18 <col:20> 'int' lvalue ParmVar
0x57cc6aaa8700 'a' 'int'

|       |   `-IntegerLiteral 0x57cc6aaa8a38 <col:24> 'int' 2

|       `-ParenExpr 0x57cc6aaa8b28 <col:29, col:35> 'int'

|         `-BinaryOperator 0x57cc6aaa8b08 <col:30, col:34> 'int' '+'

|           |-ImplicitCastExpr 0x57cc6aaa8af0 <col:30> 'int' <LValueToRValue>

|           | `-DeclRefExpr 0x57cc6aaa8ab0 <col:30> 'int' lvalue ParmVar
0x57cc6aaa8780 'b' 'int'

|           `-IntegerLiteral 0x57cc6aaa8ad0 <col:34> 'int' 1

`-FunctionDecl 0x57cc6aaa8bf8 <line:5:1, line:7:1> line:5:5 main 'int ()'

  `-CompoundStmt 0x57cc6aaa8cd0 <col:11, line:7:1>

    `-ReturnStmt 0x57cc6aaa8cc0 <line:6:2, col:9>

      `-IntegerLiteral 0x57cc6aaa8ca0 <col:9> 'int' 0
```

Q1) FunctionDecl – what information does it carry about ternary example?

Ans 1) `|-FunctionDecl 0x57cc6aaa8868 <input.c:1:1, line:3:1> line:1:5 ternary_example 'int (int, int)'` carries the following information: It's definition in the file input.c lies between line 1, column 1 and implies line 3, column 1 (`<input.c:1:1, line:3:1>`). The name of the function starts at line 1, column 5 (`line:1:5`). The function takes two arguments, both int and returns an int value (`'int (int, int)'`).

Q2) ParmVarDecl – how are the parameters a and b recorded?

Ans 2) `| |-ParmVarDecl 0x57cc6aaa8700 <col:22, col:26> col:26 used a 'int'`

`| |-ParmVarDecl 0x57cc6aaa8780 <col:29, col:33> col:33 used b 'int'`

The parameters a and b are represented as ParmVarDecl in the Abstract Syntax Tree (AST) which stores the parameter's name, type, source location. Ex: Here a, b are recorded as integer parameters for the function ternary_example. The used keyword indicates that the parameters are used inside the function.

Q3) ConditionalOperator – which child nodes correspond to the condition, the true branch, and the false branch?

Ans 3) In `|  `-ConditionalOperator 0x57cc6aaa8b48 <col:9, col:35> 'int'`, the True branch (a * 2) is

```
|       |-ParenExpr 0x57cc6aaa8a90 <col:19, col:25> 'int'
|       | `-BinaryOperator 0x57cc6aaa8a70 <col:20, col:24> 'int' '*'
|       |   |-ImplicitCastExpr 0x57cc6aaa8a58 <col:20> 'int' <LValueToRValue>
|       |   | `-DeclRefExpr 0x57cc6aaa8a18 <col:20> 'int' lvalue ParmVar
0x57cc6aaa8700 'a' 'int'
|       |   `-IntegerLiteral 0x57cc6aaa8a38 <col:24> 'int' 2
```

The False branch (b + 1) is:

```
|       `-ParenExpr 0x57cc6aaa8b28 <col:29, col:35> 'int'
|         `-BinaryOperator 0x57cc6aaa8b08 <col:30, col:34> 'int' '+'
|           |-ImplicitCastExpr 0x57cc6aaa8af0 <col:30> 'int' <LValueToRValue>
|           | `-DeclRefExpr 0x57cc6aaa8ab0 <col:30> 'int' lvalue ParmVar
0x57cc6aaa8780 'b' 'int'
```

Q4) BinaryOperator – identify the node for a > b and state how the operator kind and operand types are represented.

Ans 4) Node for a > b: `|| `-BinaryOperator 0x57cc6aaa89d8 <col:10, col:14> 'int' '>'|`

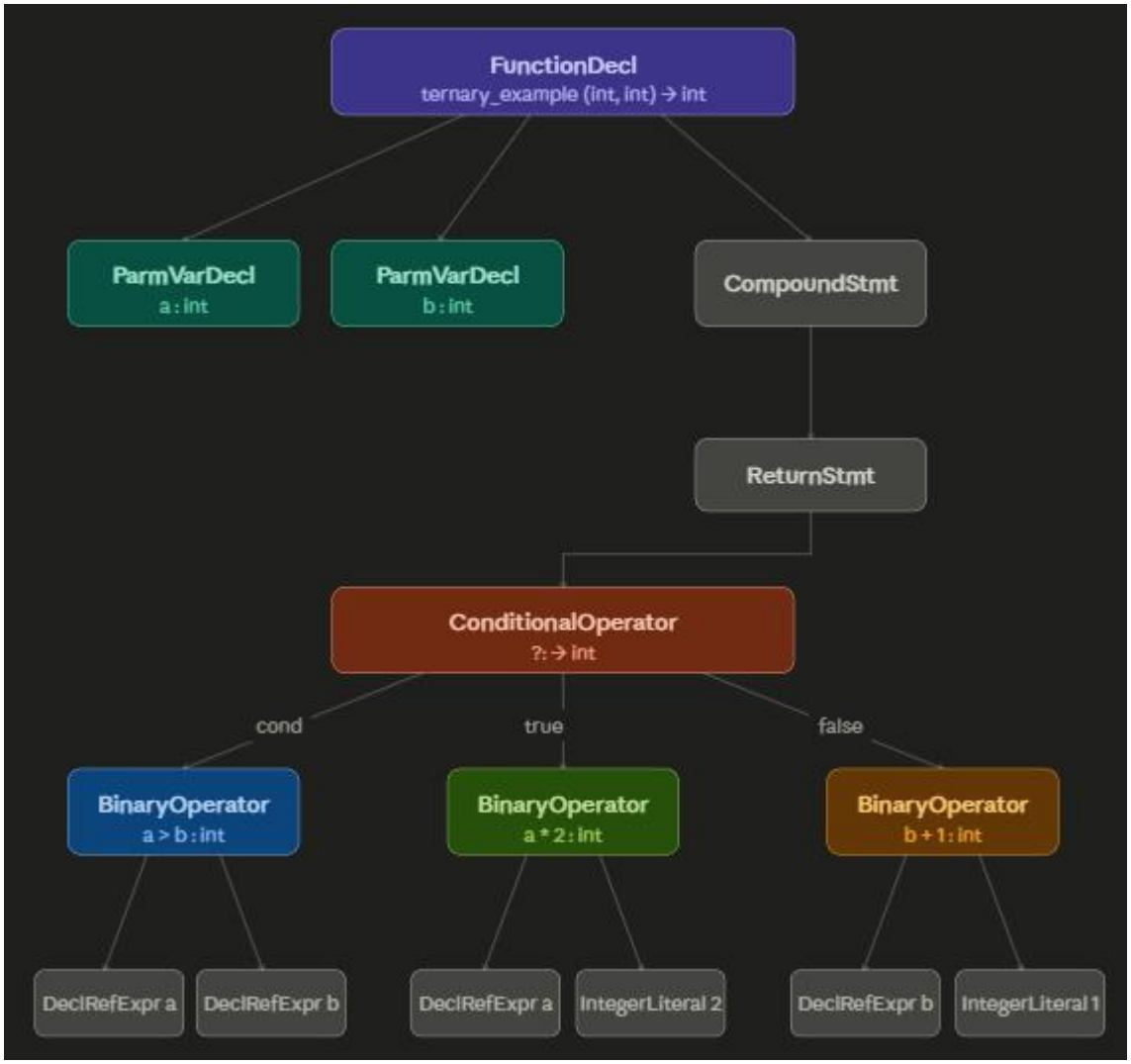
The operand kind is represented as > indicating a greater than comparison operator. The result type of the expression is int (Boolean like result).

```
|   | `-DeclRefExpr 0x57cc6aaa8968 <col:10> 'int' lvalue ParmVar 0x57cc6aaa8700
'a' 'int'
|   |  `-DeclRefExpr 0x57cc6aaa8988 <col:14> 'int' lvalue ParmVar 0x57cc6aaa8780
'b' 'int'
```

The operands of the operator are represented as child nodes, which are the DeclRefExpr nodes (see above) referring to the variables a and b.

Q5) Draw a tree diagram of the AST for the function body only, using the node names from the --ast-dump output as labels.

Ans 5) PTO



B)

Q1) How does the ConditionalOperator node in the AST map to IR instructions at -O0? Trace the path from AST node to IR.

Ans 1) Corresponding IR for the AST for Ans 3 of Q3A:

```
%7 = icmp sgt i32 %5, %6                ; Compare a and b
br i1 %7, label %8, label %11           ; If a > b, branch to label 8, else label
11
8:                                      ; Label 8, If a > b branch
%10 = mul nsw i32 %9, 2                 ; a = a * 2
br label %14                            ; Branch to label 14, end of If

11:                                     ; Label 11, else branch
%13 = add nsw i32 %12, 1                ; b = b + 1
br label %14                            ; Branch to label 14, end of If

14:                                     ; Label 14, end of If
%15 = phi i32 [ %10, %8 ], [ %13, %11 ] ; phi instruction chooses between
returning a * 2 or b + 1 based on the if condition.
```

The ConditionalOperator in the AST is translated into an explicit control flow at -O0. Condition (a > b): Implemented using icmp, followed by a conditional branch (br). The true and false expressions are computed in separate basic blocks, and a phi instruction merges the results at the join point.

Q2) At -O1, does the ternary reduce to a select instruction? If so, explain what select does and why it is a valid replacement for the branching version.

Ans 2) ternary_O1.ll:

```
%3 = icmp sgt i32 %0, %1          ; Compare a and b
%4 = shl nsw i32 %0, 1            ; a = a << 1 = a * 2
%5 = add nsw i32 %1, 1            ; b = b + 1
%6 = select i1 %3, i32 %4, i32 %5 ; Select between a * 2, b + 1
ret i32 %6                        ; Return the selected value
```

Yes, the ternary reduces to a select instruction at -O1 level of optimization. The select instruction selects the value (either %4 or %5) based on the result of the if condition. This is valid as the ternary operation essentially represents a value selection rather than a control flow. The use of select avoids branching, improves performance by enabling better instruction level parallelism.

Q3) Provide five C program examples that can demonstrate the difference in O0 and O1.ll files and point out the name of the applied optimization(s) and explain your understanding of the optimization(s) for each example.

Ans 3)

1) Constant Folding.
C code: `int x = 4 + 5;`
Optimization: We directly compute it at compile time as 9.

2) Dead Code elimination.
C code: `int x = 5;`
`x = 2;`
Optimization: We ignore the first assignment.

3) Strength Reduction
C code: `int x = b * 8;`
IR: `shl i32 %b, 3`
Optimization: Converts a multiplication instruction to a shift left (shl) instruction reducing the effective number of instructions in assembly/IR.

4) Common Subexpression Elimination (CSE).
C code: `int x = a;`
`int y = a;`
Optimization: We avoid repeated loads by detecting identical expressions that are computed multiple times and reuse the same result at appropriate places instead of recalculating.

5) Branch Elimination.
C code: `return (a > b) ? a : b;`

IR (Optimized): `select i1 ... ; We convert a ternary operator to a select instruction. Reduces branching and improves efficiency of IR code.`

Declaration of usage of AI/LLMs

I declare that I have not used any AI/LLM tools to copy, cheat, plagiarize content, or obtain solutions from other students. AI tools were used only for limited assistance in troubleshooting installation issues, clarifying compiler concepts, understanding LLVM IR syntax, checking whether obtained results were reasonable, and assisting in AST generation/visualization. AI tools were also used for minor language polishing, without altering the technical content. All implementations, code, analyses, interpretations, and explanations submitted in this report are my own original work.

Source Code Appendices

1) Q1a.ll

```
; ModuleID = 'Q1a.c'

source_filename = "Q1a.c"

target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-
n8:16:32:64-S128"

target triple = "x86_64-pc-linux-gnu"


@a = dso_local global i32 5, align 4


; Function Attrs: noinline nounwind optnone uwtable

define dso_local i32 @main() #0 {

  %1 = alloca i32, align 4

  %2 = alloca i32, align 4

  %3 = alloca i32, align 4

  store i32 10, ptr %1, align 4

  store i32 3, ptr %2, align 4

  %4 = load i32, ptr @a, align 4

  %5 = load i32, ptr %1, align 4

  %6 = load i32, ptr %2, align 4

  %7 = mul nsw i32 %5, %6

  %8 = add nsw i32 %4, %7

  store i32 %8, ptr %3, align 4

  ret i32 0

}


attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-
legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-
size"="8" "target-cpu"="x86-64" "target-
features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }


!llvm.module.flags = !{!0, !1, !2, !3, !4}

!llvm.ident = !{!5}
```

```
!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{i32 7, !"frame-pointer", i32 2}
!5 = !{!"Ubuntu clang version 17.0.6 (9ubuntu1)"}
```

2) Q1b.ll

```
; ModuleID = 'Q1b.c'
source_filename = "Q1b.c"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-
n8:16:32:64-S128"
target triple = "x86_64-pc-linux-gnu"

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() #0 {
  %1 = alloca i32, align 4
  %2 = alloca i32, align 4
  %3 = alloca i32, align 4
  %4 = alloca i32, align 4
  store i32 0, ptr %1, align 4
  store i32 2, ptr %2, align 4
  store i32 3, ptr %3, align 4
  %5 = load i32, ptr %2, align 4
  %6 = icmp eq i32 %5, 2
  br i1 %6, label %7, label %8

7:                                                ; preds = %0
  store i32 2, ptr %2, align 4
  br label %9

8:                                                ; preds = %0
  store i32 2, ptr %2, align 4
  br label %9
```

```
9:                                                ; preds = %8, %7
  %10 = load i32, ptr %2, align 4
  switch i32 %10, label %13 [
    i32 1, label %11
    i32 2, label %12
  ]

11:                                               ; preds = %9
  store i32 2, ptr %2, align 4
  br label %14

12:                                               ; preds = %9
  store i32 2, ptr %2, align 4
  br label %14

13:                                               ; preds = %9
  store i32 2, ptr %2, align 4
  br label %14

14:                                               ; preds = %13, %12, %11
  %15 = load i32, ptr %2, align 4
  store i32 %15, ptr %4, align 4
  br label %16

16:                                               ; preds = %22, %14
  %17 = load i32, ptr %4, align 4
  %18 = icmp slt i32 %17, 5
  br i1 %18, label %19, label %25

19:                                               ; preds = %16
  %20 = load i32, ptr %2, align 4
  %21 = add nsw i32 %20, 1
  store i32 %21, ptr %2, align 4
```

```
  br label %22

22:                                               ; preds = %19
  %23 = load i32, ptr %4, align 4
  %24 = add nsw i32 %23, 1
  store i32 %24, ptr %4, align 4
  br label %16, !llvm.loop !6

25:                                               ; preds = %16
  br label %26

26:                                               ; preds = %29, %25
  %27 = load i32, ptr %2, align 4
  %28 = icmp slt i32 %27, 5
  br i1 %28, label %29, label %32

29:                                               ; preds = %26
  %30 = load i32, ptr %2, align 4
  %31 = add nsw i32 %30, 1
  store i32 %31, ptr %2, align 4
  br label %26, !llvm.loop !8

32:                                               ; preds = %26
  br label %33

33:                                               ; preds = %36, %32
  %34 = load i32, ptr %2, align 4
  %35 = add nsw i32 %34, 1
  store i32 %35, ptr %2, align 4
  br label %36

36:                                               ; preds = %33
  %37 = load i32, ptr %2, align 4
  %38 = icmp slt i32 %37, 5
```

```
  br i1 %38, label %33, label %39, !llvm.loop !9


39:                                               ; preds = %36
  ret i32 0
}


attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-
legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-
size"="8" "target-cpu"="x86-64" "target-
features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }


!llvm.module.flags = !{!0, !1, !2, !3, !4}
!llvm.ident = !{!5}


!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{i32 7, !"frame-pointer", i32 2}
!5 = !{!"Ubuntu clang version 17.0.6 (9ubuntu1)"}
!6 = distinct !{!6, !7}
!7 = !{!"llvm.loop.mustprogress"}
!8 = distinct !{!8, !7}
!9 = distinct !{!9, !7}
```

3) Q1c.ll

```
; ModuleID = 'Q1c.c'
source_filename = "Q1c.c"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-
n8:16:32:64-S128"
target triple = "x86_64-pc-linux-gnu"


@.str = private unnamed_addr constant [21 x i8] c"The square of 3 = %d\00", align
1


; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @square(i32 noundef %0) #0 {
```

```
  %2 = alloca i32, align 4
  store i32 %0, ptr %2, align 4
  %3 = load i32, ptr %2, align 4
  %4 = load i32, ptr %2, align 4
  %5 = mul nsw i32 %3, %4
  ret i32 %5
}

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() #0 {
  %1 = alloca i32, align 4
  store i32 0, ptr %1, align 4
  %2 = call i32 @square(i32 noundef 3)
  %3 = call i32 (ptr, ...) @printf(ptr noundef @.str, i32 noundef %2)
  ret i32 0
}

declare i32 @printf(ptr noundef, ...) #1

attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-
legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-
size"="8" "target-cpu"="x86-64" "target-
features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
attributes #1 = { "frame-pointer"="all" "no-trapping-math"="true" "stack-
protector-buffer-size"="8" "target-cpu"="x86-64" "target-
features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

!llvm.module.flags = !{!0, !1, !2, !3, !4}
!llvm.ident = !{!5}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{i32 7, !"frame-pointer", i32 2}
!5 = !{!"Ubuntu clang version 17.0.6 (9ubuntu1)"}
```

4) Q1d.ll

```
; ModuleID = 'Q1d.c'
source_filename = "Q1d.c"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-
n8:16:32:64-S128"
target triple = "x86_64-pc-linux-gnu"

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() #0 {
  %1 = alloca i32, align 4
  %2 = alloca i32, align 4
  %3 = alloca double, align 8
  %4 = alloca i32, align 4
  %5 = alloca i64, align 8
  store i32 0, ptr %1, align 4
  store i32 1, ptr %2, align 4
  store double 2.000000e+00, ptr %3, align 8
  %6 = load double, ptr %3, align 8
  %7 = fptosi double %6 to i32
  store i32 %7, ptr %2, align 4
  store i32 3, ptr %4, align 4
  store i64 4, ptr %5, align 8
  %8 = load i32, ptr %4, align 4
  %9 = sext i32 %8 to i64
  store i64 %9, ptr %5, align 8
  ret i32 0
}

attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-
legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-
size"="8" "target-cpu"="x86-64" "target-
features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

!llvm.module.flags = !{!0, !1, !2, !3, !4}
!llvm.ident = !{!5}
```

```
!0 = !{i32 1, !"wchar_size", i32 4}

!1 = !{i32 8, !"PIC Level", i32 2}

!2 = !{i32 7, !"PIE Level", i32 2}

!3 = !{i32 7, !"uwtable", i32 2}

!4 = !{i32 7, !"frame-pointer", i32 2}

!5 = !{!"Ubuntu clang version 17.0.6 (9ubuntu1)"}
```

5) Q2a.ll

```
; ModuleID = 'Q2a.c'

source_filename = "Q2a.c"

target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-
n8:16:32:64-S128"

target triple = "x86_64-pc-linux-gnu"


; Function Attrs: noinline nounwind optnone uwtable

define dso_local i32 @main() #0 {

  %1 = alloca i32, align 4

  %2 = alloca [5 x i32], align 16

  %3 = alloca ptr, align 8

  %4 = alloca i32, align 4

  %5 = alloca i32, align 4

  %6 = alloca i32, align 4

  %7 = alloca i32, align 4

  store i32 0, ptr %1, align 4

  %8 = call noalias ptr @malloc(i64 noundef 20) #2

  store ptr %8, ptr %3, align 8

  store i32 0, ptr %4, align 4

  br label %9


9:                                                ; preds = %17, %0

  %10 = load i32, ptr %4, align 4

  %11 = icmp slt i32 %10, 5

  br i1 %11, label %12, label %20
```

```
12:                                               ; preds = %9
  %13 = load i32, ptr %4, align 4
  %14 = load i32, ptr %4, align 4
  %15 = sext i32 %14 to i64
  %16 = getelementptr inbounds [5 x i32], ptr %2, i64 0, i64 %15
  store i32 %13, ptr %16, align 4
  br label %17

17:                                               ; preds = %12
  %18 = load i32, ptr %4, align 4
  %19 = add nsw i32 %18, 1
  store i32 %19, ptr %4, align 4
  br label %9, !llvm.loop !6

20:                                               ; preds = %9
  %21 = getelementptr inbounds [5 x i32], ptr %2, i64 0, i64 2
  %22 = load i32, ptr %21, align 8
  store i32 %22, ptr %5, align 4
  store i32 0, ptr %6, align 4
  br label %23

23:                                               ; preds = %32, %20
  %24 = load i32, ptr %6, align 4
  %25 = icmp slt i32 %24, 5
  br i1 %25, label %26, label %35

26:                                               ; preds = %23
  %27 = load i32, ptr %6, align 4
  %28 = load ptr, ptr %3, align 8
  %29 = load i32, ptr %6, align 4
  %30 = sext i32 %29 to i64
  %31 = getelementptr inbounds i32, ptr %28, i64 %30
  store i32 %27, ptr %31, align 4
  br label %32
```

```
32:                                               ; preds = %26
  %33 = load i32, ptr %6, align 4
  %34 = add nsw i32 %33, 1
  store i32 %34, ptr %6, align 4
  br label %23, !llvm.loop !8

35:                                               ; preds = %23
  %36 = load ptr, ptr %3, align 8
  %37 = getelementptr inbounds i32, ptr %36, i64 2
  %38 = load i32, ptr %37, align 4
  store i32 %38, ptr %7, align 4
  ret i32 0
}

; Function Attrs: nounwind allocsize(0)
declare noalias ptr @malloc(i64 noundef) #1

attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-
legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-
size"="8" "target-cpu"="x86-64" "target-
features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
attributes #1 = { nounwind allocsize(0) "frame-pointer"="all" "no-trapping-
math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-
features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
attributes #2 = { nounwind allocsize(0) }

!llvm.module.flags = !{!0, !1, !2, !3, !4}
!llvm.ident = !{!5}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{i32 7, !"frame-pointer", i32 2}
!5 = !{!"Ubuntu clang version 17.0.6 (9ubuntu1)"}
```

```
!6 = distinct !{!6, !7}

!7 = !{!"llvm.loop.mustprogress"}

!8 = distinct !{!8, !7}
```

6) Q2bi.ll

```
; ModuleID = 'Q2bi.c'

source_filename = "Q2bi.c"

target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-
n8:16:32:64-S128"

target triple = "x86_64-pc-linux-gnu"


%struct.Point = type { i32, i32 }


; Function Attrs: noinline nounwind optnone uwtable

define dso_local i32 @main() #0 {

  %1 = alloca i32, align 4

  %2 = alloca %struct.Point, align 4

  store i32 0, ptr %1, align 4

  %3 = getelementptr inbounds %struct.Point, ptr %2, i32 0, i32 0

  store i32 1, ptr %3, align 4

  %4 = getelementptr inbounds %struct.Point, ptr %2, i32 0, i32 1

  store i32 2, ptr %4, align 4

  ret i32 0

}


attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-
legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-
size"="8" "target-cpu"="x86-64" "target-
features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }


!llvm.module.flags = !{!0, !1, !2, !3, !4}

!llvm.ident = !{!5}


!0 = !{i32 1, !"wchar_size", i32 4}

!1 = !{i32 8, !"PIC Level", i32 2}

!2 = !{i32 7, !"PIE Level", i32 2}
```

```
!3 = !{i32 7, !"uwtable", i32 2}

!4 = !{i32 7, !"frame-pointer", i32 2}

!5 = !{!"Ubuntu clang version 17.0.6 (9ubuntu1)"}
```

7) Q2bii.ll

```
; ModuleID = 'Q2bii.cpp'

source_filename = "Q2bii.cpp"

target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-
n8:16:32:64-S128"

target triple = "x86_64-pc-linux-gnu"


%class.Point = type { i32, i32 }


$_ZN5PointC2Ev = comdat any


; Function Attrs: mustprogress noinline norecurse optnone uwtable

define dso_local noundef i32 @main() #0 {

  %1 = alloca i32, align 4

  %2 = alloca %class.Point, align 4

  store i32 0, ptr %1, align 4

  call void @_ZN5PointC2Ev(ptr noundef nonnull align 4 dereferenceable(8) %2)

  ret i32 0

}


; Function Attrs: noinline nounwind optnone uwtable

define linkonce_odr dso_local void @_ZN5PointC2Ev(ptr noundef nonnull align 4
dereferenceable(8) %0) unnamed_addr #1 comdat align 2 {

  %2 = alloca ptr, align 8

  store ptr %0, ptr %2, align 8

  %3 = load ptr, ptr %2, align 8

  %4 = getelementptr inbounds %class.Point, ptr %3, i32 0, i32 0

  store i32 0, ptr %4, align 4

  %5 = getelementptr inbounds %class.Point, ptr %3, i32 0, i32 1

  store i32 0, ptr %5, align 4

  ret void
```

```
}


attributes #0 = { mustprogress noinline norecurse optnone uwtable "frame-
pointer"="all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-
protector-buffer-size"="8" "target-cpu"="x86-64" "target-
features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

attributes #1 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-
legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-
size"="8" "target-cpu"="x86-64" "target-
features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }


!llvm.module.flags = !{!0, !1, !2, !3, !4}

!llvm.ident = !{!5}


!0 = !{i32 1, !"wchar_size", i32 4}

!1 = !{i32 8, !"PIC Level", i32 2}

!2 = !{i32 7, !"PIE Level", i32 2}

!3 = !{i32 7, !"uwtable", i32 2}

!4 = !{i32 7, !"frame-pointer", i32 2}

!5 = !{!"Ubuntu clang version 17.0.6 (9ubuntu1)"}
```

8) ternary_O0.ll

```
; ModuleID = 'input.c'

source_filename = "input.c"

target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-
f80:128-n8:16:32:64-S128"

target triple = "x86_64-pc-linux-gnu"


; Function Attrs: noinline nounwind optnone uwtable

define dso_local i32 @ternary_example(i32 noundef %0, i32 noundef %1) #0 {

  %3 = alloca i32, align 4

  %4 = alloca i32, align 4

  store i32 %0, ptr %3, align 4

  store i32 %1, ptr %4, align 4

  %5 = load i32, ptr %3, align 4

  %6 = load i32, ptr %4, align 4

  %7 = icmp sgt i32 %5, %6
```

```
  br i1 %7, label %8, label %11

8:                                                ; preds = %2
  %9 = load i32, ptr %3, align 4
  %10 = mul nsw i32 %9, 2
  br label %14

11:                                               ; preds = %2
  %12 = load i32, ptr %4, align 4
  %13 = add nsw i32 %12, 1
  br label %14

14:                                               ; preds = %11, %8
  %15 = phi i32 [ %10, %8 ], [ %13, %11 ]
  ret i32 %15
}

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() #0 {
  %1 = alloca i32, align 4
  store i32 0, ptr %1, align 4
  ret i32 0
}

attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-
legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-
size"="8" "target-cpu"="x86-64" "target-
features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

!llvm.module.flags = !{!0, !1, !2, !3, !4}
!llvm.ident = !{!5}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
```

```
!3 = !{i32 7, !"uwtable", i32 2}

!4 = !{i32 7, !"frame-pointer", i32 2}

!5 = !{!"Ubuntu clang version 18.1.3 (1ubuntu1)"}
```

9) ternary_O1.ll

```
; ModuleID = 'input.c'

source_filename = "input.c"

target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-
f80:128-n8:16:32:64-S128"

target triple = "x86_64-pc-linux-gnu"


; Function Attrs: mustprogress nofree norecurse nosync nounwind willreturn
memory(none) uwtable

define dso_local i32 @ternary_example(i32 noundef %0, i32 noundef %1)
local_unnamed_addr #0 {

  %3 = icmp sgt i32 %0, %1

  %4 = shl nsw i32 %0, 1

  %5 = add nsw i32 %1, 1

  %6 = select i1 %3, i32 %4, i32 %5

  ret i32 %6

}


; Function Attrs: mustprogress nofree norecurse nosync nounwind willreturn
memory(none) uwtable

define dso_local noundef i32 @main() local_unnamed_addr #0 {

  ret i32 0

}


attributes #0 = { mustprogress nofree norecurse nosync nounwind willreturn
memory(none) uwtable "min-legal-vector-width"="0" "no-trapping-math"="true"
"stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-
features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }


!llvm.module.flags = !{!0, !1, !2, !3}

!llvm.ident = !{!4}


!0 = !{i32 1, !"wchar_size", i32 4}
```

```
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{!"Ubuntu clang version 18.1.3 (1ubuntu1)"}
```